CHAPTER 1

INTRODUCTION

1.1 OVERVIEW

Line following robots are one of the most fundamental and widely used applications in the field of robotics and automation. These robots are designed to follow a predefined path, typically a black line on a white surface, using optical sensors. The concept finds extensive use in industrial automation, warehouse logistics, automated guided vehicles (AGVs), and educational robotics.

A line follower robot is an autonomous vehicle that detects and follows a line drawn on the floor. The path is usually a black line on a white surface (or vice versa) with high contrast between the line and the background surface. The robot uses infrared (IR) sensors to detect the line and adjusts its movement accordingly to stay on the path.

The principle of operation is based on the difference in light reflectivity between dark and light surfaces. Black surfaces absorb infrared light, while white surfaces reflect it. IR sensors detect this difference and provide digital signals to the microcontroller, which then controls the motors to keep the robot on track.

1.2 BACKGROUND AND MOTIVATION

Autonomous navigation is a key area of research and development in robotics. Line following robots represent the simplest form of autonomous guided vehicles (AGVs) that are widely used in industrial settings for material handling and transportation.

The motivation behind this project includes:

- Understanding the fundamentals of autonomous navigation
- Learning sensor integration and calibration techniques
- Gaining hands-on experience with motor control systems
- Developing embedded programming skills
- Creating a platform for more advanced robotics projects

1.3 PROBLEM STATEMENT

In many industrial and commercial environments, there is a need for automated material transport along predefined paths. Manual transportation is inefficient, prone to human error, and costly. A cost-effective, reliable line following robot can automate this process.

The challenge lies in designing a robot that can:

- Accurately detect and follow a line at reasonable speeds
- Handle curves, turns, and path interruptions
- Provide feedback about its operational state
- Be built using readily available, low-cost components

- Be easily modifiable and extensible

1.4 OBJECTIVES

The primary objectives of this project are:

1. To design a 4WD robot chassis capable of following a black line on a white surface.
2. To integrate TCRT5000 IR sensors for accurate and reliable line detection.
3. To program an Arduino Uno microcontroller to process sensor data and control motors accordingly.
4. To implement a robust line-following algorithm with real-time decision making.
5. To provide real-time audio-visual feedback using LEDs and a buzzer for different operational states.
6. To create a well-documented, modular project suitable for learning and future enhancements.

1.5 ORGANIZATION OF THE REPORT

This report is organized into ten chapters. Chapter 2 presents a literature survey. Chapter 3 describes the system design and block diagram. Chapter 4 details hardware components. Chapter 5 covers circuit diagram and wiring. Chapter 6 explains the software. Chapter 7 presents flowchart and algorithm. Chapter 8 discusses test results. Chapter 9 lists applications, advantages, and disadvantages. Chapter 10 concludes with future scope.

1.6 SUMMARY

This chapter introduced the concept of line following robots, the motivation behind this project, the problem statement, and the objectives. The line follower robot demonstrates sensor integration, microcontroller programming, and autonomous navigation principles.

CHAPTER 2

LITERATURE SURVEY

2.1 EXISTING LINE FOLLOWING ROBOT SYSTEMS

Line following robots have been extensively studied for applications ranging from educational platforms to industrial AGVs. Several approaches exist for line detection.

2.1.1 Single Sensor Line Followers

The simplest line following robots use a single IR sensor with an edge-following algorithm. Limitations include inability to detect line loss, poor performance on sharp curves, and limited speed.

2.1.2 Two Sensor Line Followers

The two-sensor configuration is the most common approach. Two IR sensors placed side-by-side allow the robot to detect which side is deviating from the line and make corrections. The decision logic is straightforward:

- Both sensors on black → Move forward
- Left on white, right on black → Turn left
- Left on black, right on white → Turn right
- Both on white → Line lost, stop

2.1.3 Multi-Sensor Arrays

Advanced line followers use 5-8 IR sensors for higher resolution line position detection, smoother PID control, and higher speeds. However, this increases cost and complexity.

2.1.4 Camera-Based Line Detection

Camera-based systems offer long-range detection and complex path handling but require significant processing power and complex algorithms.

2.2 COMPARISON OF APPROACHES

Feature	Single Sensor	Dual Sensor	Multi-Sensor	Camera
Cost	Very Low	Low	Medium	High
Complexity	Simple	Simple	Moderate	High
Curve Handling	Poor	Good	Excellent	Excellent
Speed	Low	Medium	High	Very High
Suitable For	Learning	Projects	Advanced	Research

2.3 OUR APPROACH

For this project, we selected the two-sensor approach using TCRT5000 IR sensors, providing a good balance between cost and performance with reliable line following capability and a modular design for future upgrades.

2.4 SUMMARY

The literature survey confirms that the dual IR sensor approach is well-suited for educational line following robot projects, providing reliable operation while maintaining simplicity and low cost.

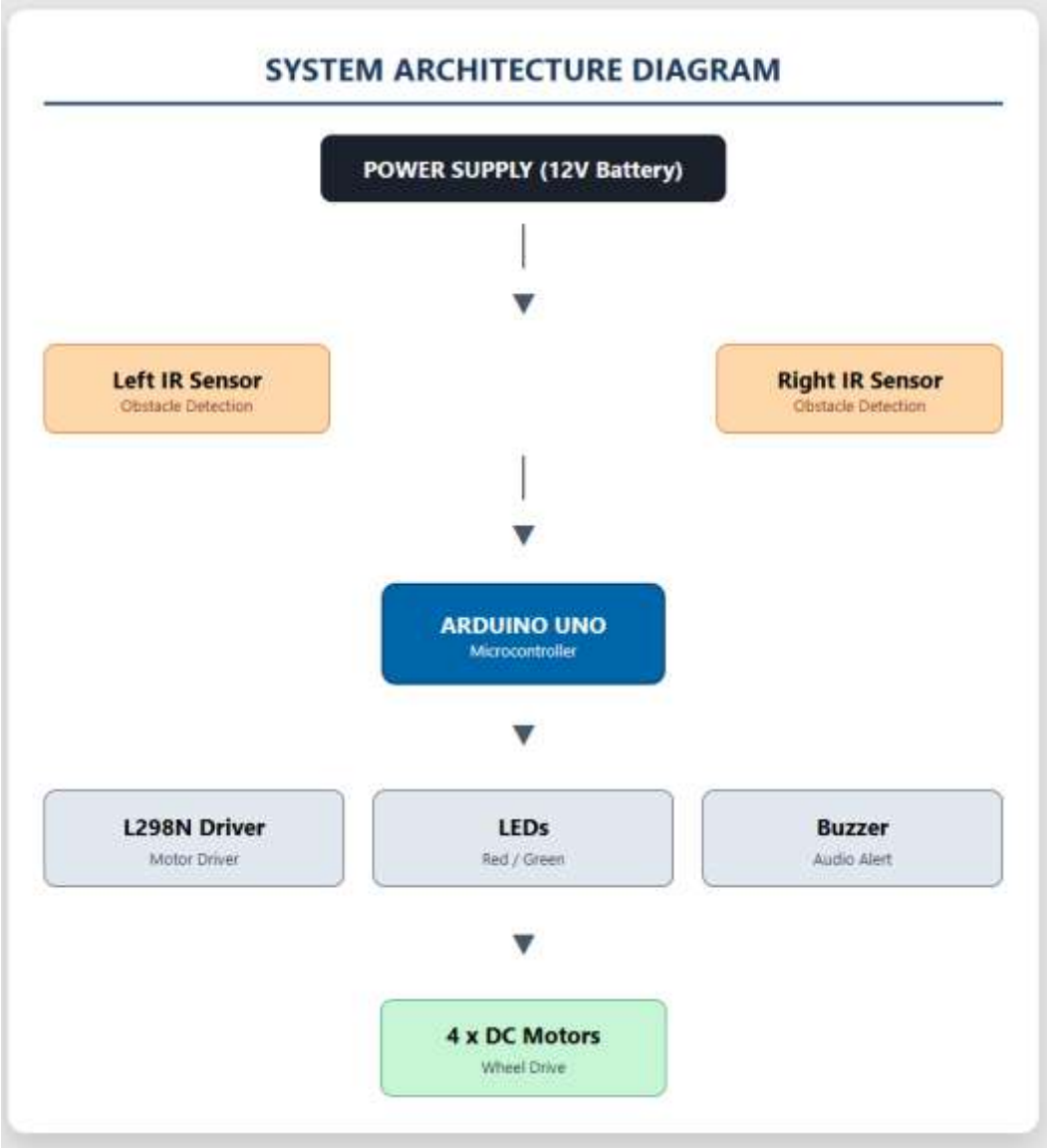
CHAPTER 3

SYSTEM DESIGN AND OPERATION

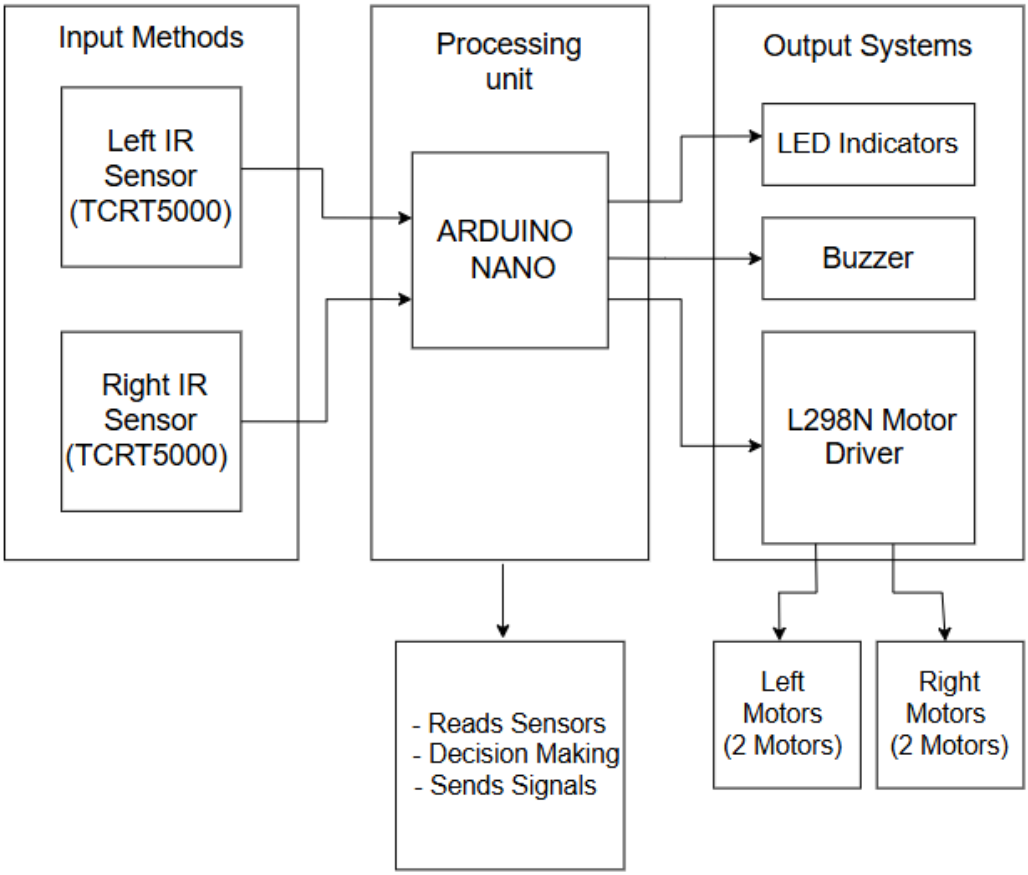
3.1 SYSTEM ARCHITECTURE

The line following robot consists of four main sections:

1. **Input Section:** Two TCRT5000 IR sensors that detect the line
2. **Processing Section:** Arduino Uno that processes sensor data and makes decisions
3. **Output Section:** L298N motor driver, DC motors, LEDs, and buzzer
4. **Power Section:** 12V rechargeable battery



3.2 BLOCK DIAGRAM



3.3 WORKING MECHANISM

The robot's working is based on the difference in reflectivity between black and white surfaces.

3.3.1 Sensor Operation

- **White Surface:** High reflectivity → IR light reflects → Phototransistor conducts → Output = LOW (0)
- **Black Surface:** Low reflectivity → IR light absorbed → Output = HIGH (1)

Optimal sensor height: approximately 1 cm from the ground.

3.3.2 Decision Logic

SENSOR LOGIC TABLE — LINE FOLLOWING ROBOT

Left Sensor	Right Sensor	Robot Action	LED Feedback	Buzzer
1 (Black)	1 (Black)	Move Forward	Green ON	No Sound
1 (Black)	0 (White)	Turn Left	Red ON	Short Beep
0 (White)	1 (Black)	Turn Right	Red ON	Short Beep
0 (White)	0 (White)	Stop	Both Blinking	Beeping

Note: 1 = Black line detected, 0 = White surface detected

3.3.3 Motor Control Logic

ROBOT MOTOR CONTROL LOGIC

Robot Action	Left Motors	Right Motors
Forward	Forward	Forward
Turn Left	Stop	Forward
Turn Right	Forward	Stop
Stop	Stop	Stop

Note: Left and Right motors operate independently based on sensor input for precise line following.

Speed is controlled via PWM on ENA (Pin 5) and ENB (Pin 9).

3.4 SUMMARY

This chapter presented the system architecture, block diagram, and working mechanism. The dual IR sensor approach with a simple decision-making algorithm provides reliable line following capability.

CHAPTER 4

HARDWARE DESIGN AND DESCRIPTION

4.1 ARDUINO UNO

The Arduino Uno is based on the ATmega328P microcontroller and serves as the brain of the robot.

Specifications:

- Microcontroller: ATmega328P
- Operating Voltage: 5V
- Input Voltage: 7-12V
- Digital I/O Pins: 14 (6 PWM)
- Analog Input Pins: 6
- Flash Memory: 32 KB
- SRAM: 2 KB
- Clock Speed: 16 MHz

4.2 TCRT5000 IR SENSOR

The TCRT5000 is a reflective optical sensor with an infrared emitter and phototransistor.

Specifications:

- Operating Voltage: 5V DC
- Detection Range: 1mm to 15mm
- Output Type: Digital (LOW = reflected, HIGH = not reflected)
- On-board LM393 comparator
- Adjustable sensitivity via potentiometer

4.3 L298N MOTOR DRIVER

The L298N is a dual H-bridge motor driver for controlling two DC motors independently.

Specifications:

- Driver Chip: L298N
- Motor Supply: 5V to 35V (12V used)
- Logic Supply: 5V
- Max Current: 2A per channel
- Control: Forward, Reverse, Stop, Brake

- Built-in 5V regulator

4.4 DC GEARED MOTORS

Four identical DC geared motors in 4WD configuration.

Specifications:

- Operating Voltage: 3V to 12V
- Rated Speed: 100-300 RPM
- Gear Ratio: 48:1
- Stall Torque: 0.8 kg-cm

The 4WD configuration provides better traction, stability, and payload capacity.

4.5 LEDS AND BUZZER

- **Red LED:** Pin 12 (via 220Ω) — Turn/Error indication
- **Green LED:** Pin 13 (via 220Ω) — Forward indication
- **Buzzer:** Pin 11 — Audio feedback for turns and line loss

4.6 HARDWARE COMPONENTS SUMMARY

#	Component	Qty	Purpose
1	Arduino Uno	1	Main Controller
2	TCRT5000 IR Sensor	2	Line Detection
3	L298N Motor Driver	1	Motor Control
4	DC Geared Motors	4	Drive Wheels
5	Wheels (65mm)	4	Movement
6	Ball Caster	1	Balance
7	12V Li-ion Battery	1	Power Source

#	Component	Qty	Purpose
8	Red LED	1	Turn/Error Indication
9	Green LED	1	Forward Indication
10	Buzzer	1	Audio Feedback
11	220Ω Resistors	2	LED Current Limiting
12	4WD Acrylic Chassis	1	Robot Frame
13	Jumper Wires	20+	Connections

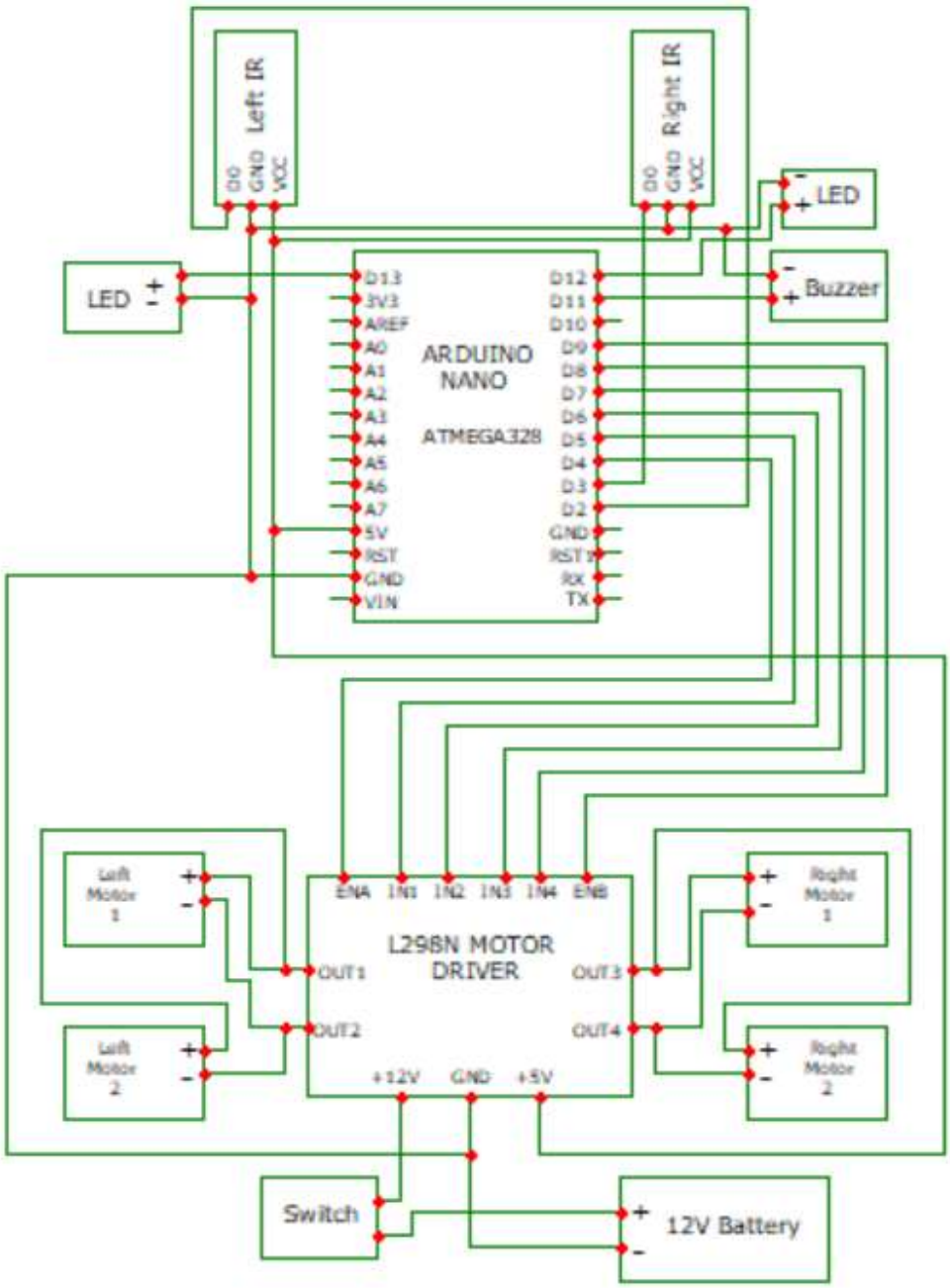
4.7 SUMMARY

Each component was selected based on availability, cost, compatibility, and suitability for the line following robot application. The 4WD configuration with L298N provides reliable motor control with audio-visual feedback.

CHAPTER 5

HARDWARE IMPLEMENTATION

5.1 CIRCUIT DIAGRAM



5.2 PIN CONFIGURATION AND WIRING

5.2.1 Arduino to L298N Motor Driver

Arduino Pin	L298N Pin	Description
D5 (PWM)	ENA	Left motors speed control
D6	IN1	Left motors direction 1
D7	IN2	Left motors direction 2
D4	IN3	Right motors direction 1
D8	IN4	Right motors direction 2
D9 (PWM)	ENB	Right motors speed control
5V	+5V	Logic power

5.2.2 Arduino to IR Sensor

Component	Arduino Pin	Description
Left Sensor D0	D2	Left sensor digital output
Left Sensor VCC	5V	Sensor power
Left Sensor GND	GND	Sensor ground
Right Sensor D0	D3	Right sensor digital output

Right Sensor VCC 5V Sensor power Right Sensor GND GND Sensor ground

5.2.3 Arduino to LEDs and Buzzer

Component	Arduino Pin	Description
Red LED (Anode)	D12	Turn/Error indicator
Red LED (Cathode)	GND (via 220Ω)	Current limiting
Green LED (Anode)	D13	Forward indicator
Green LED (Cathode)	GND (via 220Ω)	Current limiting
Buzzer (+)	D11	Audio feedback
Buzzer (-)	GND	Ground

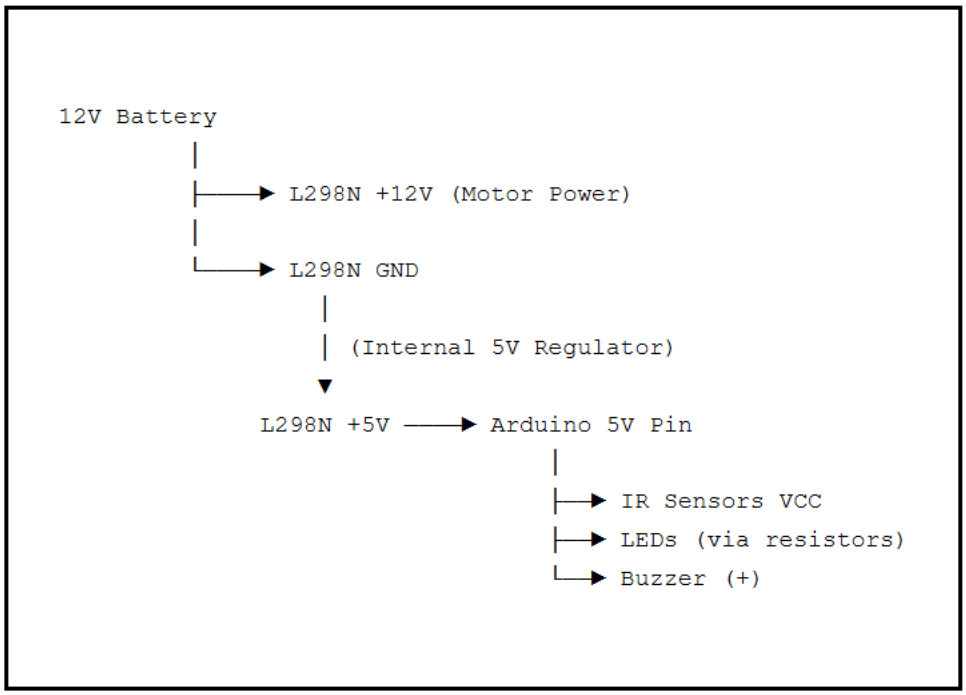
5.2.4 L298N to Motors (4WD Configuration)

L298N Terminal	Motor Connection	Description
OUT1	Left Front (+) & Left Rear (+)	Left side positive
OUT2	Left Front (-) & Left Rear (-)	Left side negative
OUT3	Right Front (+) & Right Rear (+)	Right side positive
OUT4	Right Front (-) & Right Rear (-)	Right side negative

5.2.5 L298N to Battery

L298N Terminal	Battery Connection	Description
+12V	Battery Positive	Main motor power (7V-12V)
GND	Battery Negative	Ground (Common with Arduino GND)

5.3 POWER DISTRIBUTION



5.4 ASSEMBLY STEPS

1. Mount four DC motors onto the chassis
2. Attach wheels to each motor shaft
3. Mount ball caster at front/rear
4. Secure Arduino Uno and L298N onto chassis
5. Mount IR sensors at front (15mm apart, 10mm above ground)

6. Wire all connections according to pin configuration tables
7. Double-check connections before powering ON
8. Connect 12V battery

5.5 SUMMARY

This chapter presented complete hardware implementation details including circuit diagram, pin configurations, wiring tables, power distribution, and assembly steps. All connections were verified through testing.

CHAPTER 6

SOFTWARE DESCRIPTION

6.1 ARDUINO IDE

The Arduino Integrated Development Environment (IDE) is used to write, compile, and upload code to the Arduino Uno.

Features:

- Cross-platform (Windows, Mac, Linux)
- Simple C/C++ based programming
- Built-in library manager
- Serial monitor for debugging
- One-click compilation and upload

Program Structure:

- setup(): Runs once at startup for initialization
- loop(): Runs continuously with main program logic

6.2 CODE EXPLANATION

6.2.1 Pin Definitions

cpp

```
#define LEFT_SENSOR  2
#define RIGHT_SENSOR 3
#define ENA 5
#define IN1 6
#define IN2 7
#define IN3 4
#define IN4 8
#define ENB 9
#define LED_RED   12
#define LED_GREEN 13
#define BUZZER    11
#define SPEED     90
```

6.2.2 Setup Function

The setup() function configures all pins, initializes motors to STOP state, performs a startup LED blink and buzzer beep sequence, and waits 2 seconds to place the robot on track.

6.2.3 Main Loop Function

The loop() function implements the core decision logic:

1. Read both IR sensors
2. Compare values against decision table
3. Call appropriate motor function
4. Update LED states
5. Control buzzer
6. Repeat

6.2.4 Motor Control Functions

Function	Left Motors	Right Motors	Result
forward()	`IN1=H, IN2=L, ENA=SPEED`	`IN3=H, IN4=L, ENB=SPEED`	All forward
turnLeft()	`STOP (ENA=0)`	`FORWARD`	Pivot left
turnRight()	`FORWARD`	`STOP (ENB=0)`	Pivot right
stop()	`All LOW`	`All LOW`	All stop

6.3 SOFTWARE COMPONENTS

Software	Purpose
Arduino IDE (v2.0+)	Code development and upload
Serial Monitor	Real-time debugging
Arduino C/C++	Programming language
Built-in Libraries	pinMode, digitalRead, digitalWrite, analogWrite, tone, millis

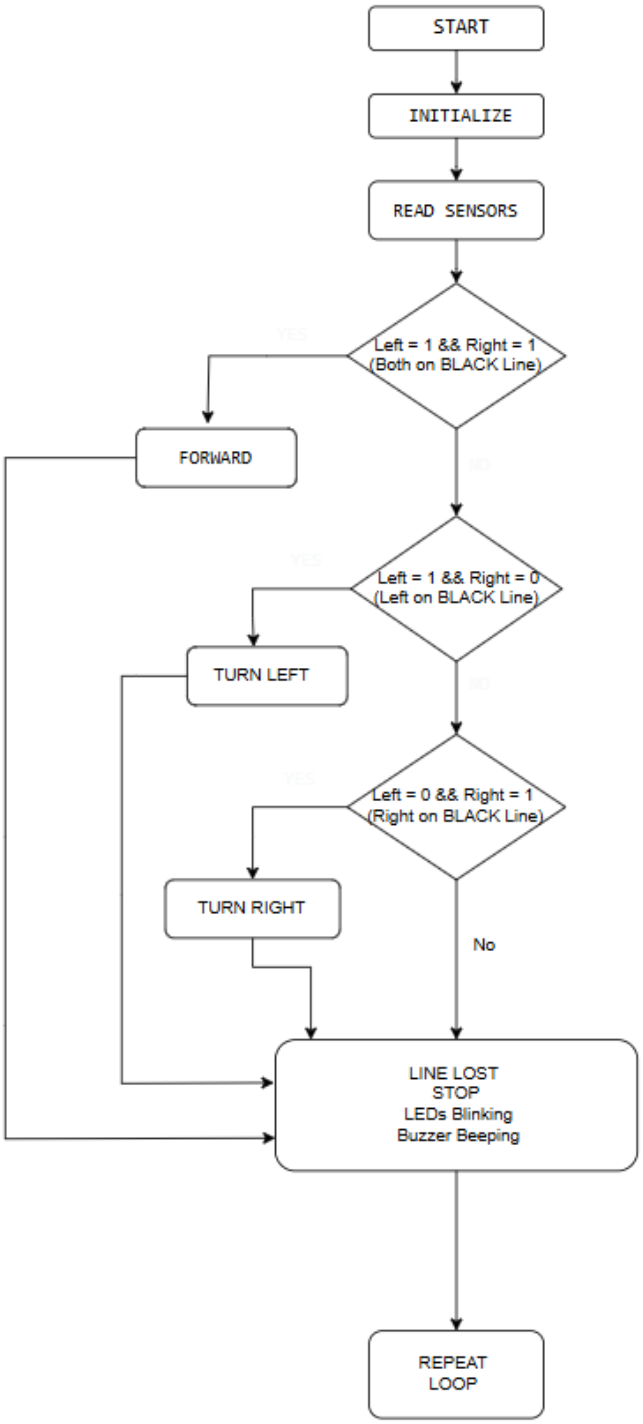
6.4 SUMMARY

This chapter described the software aspects including Arduino IDE, code structure, pin definitions, decision-making algorithm, and motor control functions.

CHAPTER 7

SOFTWARE IMPLEMENTATION

7.1 FLOWCHART



7.2 ALGORITHM

Step 1: Start the system.

Step 2: Power ON. Arduino initializes all pins and components.

Step 3: Startup sequence — blink LEDs and beep buzzer twice.

Step 4: Wait 2 seconds for robot placement on track.

Step 5: Read left IR sensor into variable left. Read right IR sensor into variable right.

Step 6: If left == 1 AND right == 1: Call forward(), Green LED ON, Red LED OFF, Buzzer OFF.

Step 7: Else if left == 1 AND right == 0: Call turnLeft(), Red LED ON, Green LED OFF, Buzzer short beep.

Step 8: Else if left == 0 AND right == 1: Call turnRight(), Red LED ON, Green LED OFF, Buzzer short beep.

Step 9: Else (both sensors on white): Call stop(), Blink both LEDs at 300ms, Beep buzzer at 500ms.

Step 10: Repeat from Step 5 continuously.

7.3 DECISION TABLE

Left Sensor	Right Sensor	Robot Action	LED Feedback	Buzzer
1 (BLACK)	1 (BLACK)	FORWARD	Green ON	No Sound
1 (BLACK)	0 (WHITE)	TURN LEFT	Red ON	Short Beep
0 (WHITE)	1 (BLACK)	TURN RIGHT	Red ON	Short Beep
0 (WHITE)	0 (WHITE)	STOP	Both Blinking	Beeping

7.4 MOTOR CONTROL LOGIC

Action	IN1	IN2	ENA	IN3	IN4	ENB
FORWARD	HIGH	LOW	SPEED	HIGH	LOW	SPEED
TURN LEFT	LOW	LOW	0	HIGH	LOW	SPEED
TURN RIGHT	HIGH	LOW	SPEED	LOW	LOW	0
STOP	LOW	LOW	0	LOW	LOW	0

7.5 SUMMARY

This chapter presented software implementation details including flowchart, algorithm, decision table, and motor control logic. The program follows a simple but effective approach for autonomous line following.

CHAPTER 8

EXPERIMENTAL RESULTS

8.1 TEST SETUP

Parameter	Value
Track Surface	White chart paper / floor
Line Color	Black electrical tape
Line Width	18mm
Sensor Height	~1cm from ground
Battery Voltage	12V (fully charged)
Motor Speed	PWM 90 (~35%)
Ambient Lighting	Normal indoor

8.2 TEST RESULTS

Test Case	Expected Result	Status
Straight line (both on black)	Move forward smoothly	✅ PASS
Left on black, right on white	Turn left on curve	✅ PASS
Right on black, left on white	Turn right on curve	✅ PASS
Sharp 90-degree turn	Follow turn without losing line	✅ PASS

Test Case	Expected Result	Status
Gradual S-curve	**Navigate both curves smoothly**	✅ **PASS**
Line interruption (gap)	**Stop and indicate line lost**	✅ **PASS**
Both sensors on white	**Stop with blinking LEDs**	✅ **PASS**
Robot placed off-track	**Stop and beep**	✅ **PASS**

8.3 OBSERVATIONS

- Robot successfully follows black lines on white surfaces with high reliability.
- Two-sensor configuration provides adequate detection for straight paths and moderate curves.
- Green LED clearly confirms forward movement. Red LED and buzzer effectively indicate turns and line loss.
- 4WD configuration provides excellent stability during turns.
- Ambient lighting affects sensor performance. Direct sunlight causes false readings.
- Sensor height is critical — optimal at 8-12mm from surface.

8.4 PERFORMANCE METRICS

Metric	Value
Line Following Accuracy	>95% on standard tracks
Maximum Curvature	90-degree turns
Response Time	~50ms
Battery Life	~45 minutes continuous
Recommended Speed	PWM 80-100
Optimal Sensor Height	8-12mm

8.5 SUMMARY

Experimental results confirm the robot successfully achieves all design objectives — reliable line following, curve handling, line loss detection, and clear audio-visual feedback.

CHAPTER 9

APPLICATIONS, ADVANTAGES AND DISADVANTAGES

9.1 APPLICATIONS

Application Area	Description
Industrial Automation	Automated Guided Vehicles (AGVs) for material transport
Educational Robotics	Teaching platform for embedded systems and control logic
Robotics Competitions	Line-following events (RoboCup, Technoxian)
Warehouse Logistics	Autonomous package delivery
Hospitality	Restaurant serving robots following floor markings
Research	Testing PID control and path planning algorithms
Healthcare	Medicine delivery robots in hospital corridors

9.2 ADVANTAGES

- **Simple Design** — Easy to build, understand, and modify
- **Low Cost** — Uses affordable, readily available components
- **Modular Architecture** — Components easily replaced or upgraded
- **Real-time Feedback** — LEDs and buzzer provide immediate status
- **4WD Stability** — Excellent traction and balance during turns
- **Customizable Speed** — PWM adjustable in code
- **Open Source** — All code and documentation freely available
- **Educational Value** — Excellent for learning embedded systems

- **Extensible** — Can be enhanced with additional sensors

9.3 DISADVANTAGES

- **Limited Path Types** — Only detects black lines on white surfaces
- **Ambient Light Sensitivity** — Performance affected by bright light
- **No Obstacle Avoidance** — Cannot detect obstacles on path
- **Battery Dependent** — Performance degrades as voltage drops
- **Fixed Sensor Position** — Cannot adapt to varying line widths
- **No Speed Feedback** — Open-loop control without encoders
- **Limited Curvature** — May struggle with very sharp turns at high speeds
- **No Wireless Control** — No remote monitoring in basic version

9.4 SUMMARY

The project demonstrates a low-cost, reliable line following robot. The advantages of simplicity, modularity, and real-time feedback outweigh the limitations for the intended educational and light industrial use cases.

CHAPTER 10

CONCLUSION AND FUTURE SCOPE

10.1 CONCLUSION

The **Autonomous 4WD Line Following Robot with Audio-Visual Feedback** was successfully designed, developed, and tested. The robot effectively follows a black line on a white surface using two TCRT5000 infrared sensors and an Arduino Uno microcontroller.

Key Achievements:

- Design and assembly of a stable 4WD robot chassis
- Integration of dual IR sensors for reliable line detection
- Implementation of a robust decision-making algorithm
- Real-time audio-visual feedback through LEDs and buzzer
- Successful navigation of straight paths, curves, and 90° turns
- Creation of comprehensive documentation for learning

The 4WD configuration with L298N motor driver provides excellent stability. The dual sensor approach offers good balance between simplicity and reliable performance.

10.2 LEARNING OUTCOMES

- Embedded C/C++ programming using Arduino IDE
- Interfacing IR sensors with microcontrollers
- Motor driver configuration and PWM speed control
- Circuit design and hardware assembly
- Sensor calibration and troubleshooting
- System integration and testing
- Technical documentation and reporting

10.3 FUTURE SCOPE

Enhancement	Description
PID Control	Smoother and faster line following
Obstacle Detection	Ultrasonic sensor for avoidance
Wireless Control	Bluetooth/Wi-Fi for remote operation
Maze Solving	Additional sensors for Micromouse
Speed Ramping	Acceleration control for smoother motion
Battery Monitoring	Voltage divider for level display
OLED Display	Speed, sensor values, status display
Computer Vision	ESP32-CAM for advanced detection
Encoder Feedback	Precise speed and distance control
IoT Connectivity	Cloud monitoring and data logging

10.4 CLOSING REMARKS

This project demonstrates that with basic components and a systematic approach, a fully functional autonomous robot can be built. The skills gained form a strong foundation for more advanced robotics and embedded systems projects.

APPENDIX

COMPLETE SOURCE CODE

```
/*
 * AUTONOMOUS 4WD LINE FOLLOWING ROBOT
 * WITH AUDIO-VISUAL FEEDBACK
 */

// ========== PIN DEFINITIONS ==========
#define LEFT_SENSOR  2
#define RIGHT_SENSOR 3
#define ENA 5
#define IN1 6
#define IN2 7
#define IN3 4
#define IN4 8
#define ENB 9
#define LED_RED   12
#define LED_GREEN 13
#define BUZZER    11
#define SPEED     90

// ========== SETUP ==========
void setup() {
  pinMode(LEFT_SENSOR, INPUT);
  pinMode(RIGHT_SENSOR, INPUT);
  pinMode(ENA, OUTPUT);
  pinMode(ENB, OUTPUT);
  pinMode(IN1, OUTPUT);
  pinMode(IN2, OUTPUT);
  pinMode(IN3, OUTPUT);
  pinMode(IN4, OUTPUT);
  pinMode(LED_RED, OUTPUT);
  pinMode(LED_GREEN, OUTPUT);
  pinMode(BUZZER, OUTPUT);

  stop();

  // Startup sequence
  digitalWrite(LED_RED, HIGH);
```

```
  digitalWrite(LED_GREEN, HIGH);
  tone(BUZZER, 1000, 200);
  delay(300);
  digitalWrite(LED_RED, LOW);
  digitalWrite(LED_GREEN, LOW);
  delay(200);
  digitalWrite(LED_RED, HIGH);
  digitalWrite(LED_GREEN, HIGH);
  tone(BUZZER, 1200, 200);
  delay(300);
  digitalWrite(LED_RED, LOW);
  digitalWrite(LED_GREEN, LOW);

  delay(2000);
}

// ========== MAIN LOOP ==========
void loop() {
  int left = digitalRead(LEFT_SENSOR);
  int right = digitalRead(RIGHT_SENSOR);

  if (left == 1 && right == 1) {
    forward();
    digitalWrite(LED_GREEN, HIGH);
    digitalWrite(LED_RED, LOW);
    noTone(BUZZER);
  }
  else if (left == 1 && right == 0) {
    turnLeft();
    digitalWrite(LED_RED, HIGH);
    digitalWrite(LED_GREEN, LOW);
    tone(BUZZER, 800, 50);
  }
  else if (left == 0 && right == 1) {
    turnRight();
    digitalWrite(LED_RED, HIGH);
    digitalWrite(LED_GREEN, LOW);
    tone(BUZZER, 1000, 50);
  }
  else {
```

```
    stop();
    static unsigned long lastBlink = 0;
    static bool blinkState = false;
    if (millis() - lastBlink > 300) {
      blinkState = !blinkState;
      digitalWrite(LED_RED, blinkState);
      digitalWrite(LED_GREEN, blinkState);
      lastBlink = millis();
    }
    static unsigned long lastBeep = 0;
    if (millis() - lastBeep > 500) {
      tone(BUZZER, 600, 100);
      lastBeep = millis();
    }
  }
}

// ========== MOTOR FUNCTIONS ==========
void forward() {
  digitalWrite(IN1, HIGH);
  digitalWrite(IN2, LOW);
  analogWrite(ENA, SPEED);
  digitalWrite(IN3, HIGH);
  digitalWrite(IN4, LOW);
  analogWrite(ENB, SPEED);
}

void turnLeft() {
  digitalWrite(IN1, LOW);
  digitalWrite(IN2, LOW);
  analogWrite(ENA, 0);
  digitalWrite(IN3, HIGH);
  digitalWrite(IN4, LOW);
  analogWrite(ENB, SPEED);
}

void turnRight() {
  digitalWrite(IN1, HIGH);
  digitalWrite(IN2, LOW);
  analogWrite(ENA, SPEED);
```

```
  digitalWrite(IN3, LOW);
  digitalWrite(IN4, LOW);
  analogWrite(ENB, 0);
}

void stop() {
  digitalWrite(IN1, LOW);
  digitalWrite(IN2, LOW);
  digitalWrite(IN3, LOW);
  digitalWrite(IN4, LOW);
  analogWrite(ENA, 0);
  analogWrite(ENB, 0);
}
```

REFERENCES

1. Arduino Official Website — Documentation and Tutorials https://www.arduino.cc/
2. L298N Dual Full-Bridge Driver Datasheet, STMicroelectronics, 2000. https://www.st.com/resource/en/datasheet/l298.pdf
3. TCRT5000 Reflective Optical Sensor Datasheet, Vishay, 2015. https://www.vishay.com/docs/83760/tcrt5000.pdf
4. ATmega328P 8-bit AVR Microcontroller Datasheet, Microchip, 2018.
5. "Embedded Systems and Robotics" — N. S. Kumar, Technical Publications, 2022.
6. "Robotics: Control, Sensing, Vision, and Intelligence" — K.S. Fu, R.C. Gonzalez, C.S.G. Lee, McGraw-Hill.